

ATIVIDADE AVALIATIVA — ESTRUTURAS DE DADOS

Parte 1: Bubble Sort e Quick Sort

Bubble Sort

- Funcionamento: compara pares vizinhos e troca se fora de ordem; repete até não haver trocas.
- Melhor caso: $O(n)$ | Caso médio: $O(n^2)$ | Pior caso: $O(n^2)$
- Vantagens: simples, in-place, estável
- Limitações: lento em arrays grandes
- Recomendado: arrays pequenos ou quase ordenados
- Não recomendado: grandes volumes de dados

Quick Sort

- Funcionamento: escolhe pivô, particiona (menores à esquerda, maiores à direita) e recursiona
- Melhor caso: $O(n \log n)$ | Caso médio: $O(n \log n)$ | Pior caso: $O(n^2)$
- Vantagens: rápido na prática, in-place
- Limitações: não estável; pior caso ruim se pivô mal escolhido
- Recomendado: uso geral, grandes volumes de dados
- Não recomendado: quando estabilidade é obrigatória

Tabela comparativa

Característica	Bubble Sort	Quick Sort
Princípio	Comparação/troca par a par	Particionamento + recursão
Melhor caso	$O(n)$	$O(n \log n)$
Caso médio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$
Memória	$O(1)$	$O(\log n)$
Vantagem	Simplicidade	Eficiência em grandes volumes
Limitação	Lento p/ arrays grandes	Pior caso ruim com pivô ruim
Uso recomendado	Arrays pequenos	Uso geral / grandes dados

Parte 2: Experimento de Ordenação

Tamanho	Bubble-Comparações	Bubble-Trocas	Quick-Comparações	Quick-Movimentações
10	45	26	29	15
20	187	114	62	44
1.000	499.329	241.624	10.695	6.335

a) Menos operações com 10 elementos? Quick Sort.

b) Comportamento igual com 20 elementos? Sim, mesmo padrão, diferença aumentando.

c) O que ocorre com 1.000 elementos? Diferença enorme: Bubble faz aproximadamente 46x mais operações.

d) Maior crescimento? Bubble Sort (quadrático).

e) Coerente com a teoria? Sim, $O(n^2)$ vs $O(n \log n)$.

f) Quando usar Bubble Sort? Arrays pequenos ou quase ordenados.

g) Quando usar Quick Sort? Uso geral, grandes volumes de dados.

Parte 3: Busca em Matrizes

Matriz	Elementos	Início	Final	Inexistente
2×2	4	1	4	4
10×10	100	1	100	100
100×100	10.000	1	10.000	10.000

a) **Por que início exige menos operações?** Encontra na 1ª comparação.

b) **Se não existe?** Percorre a matriz inteira.

c) **Pior caso?** Elemento no final ou inexistente: $m \times n$ comparações.

d) **Efeito do aumento das dimensões?** Operações crescem na mesma proporção ($m \times n$).

e) **Complexidade?** $O(m \times n)$.

Parte 4: Hands On 1: Array de Temperaturas

Temperaturas: 19.5, 21.0, 18.2, 25.3, 22.7, 30.1, 17.8, 24.4, 20.9, 23.6

- Média: 22,35 °C
- Maior: 30,1 °C (índice 5)
- Menor: 17,8 °C (índice 6)
- Acima da média: 5 valores
- Operações de percurso: 39
- Complexidade: $O(n)$, percursos independentes, sem loops aninhados

Parte 5: Hands On 2: Matriz de Sensores (5×24)

- Médias por sensor: 21.62 / 24.88 / 20.04 / 26.88 / 21.88 °C
- Maior temperatura: 35 °C (sensor 3, hora 13h)
- Média geral (120 medições): 23,06 °C
- Limite 28 °C - 24 leituras acima
- Loops aninhados: necessários (2 dimensões: sensor × hora)
- Índices $[i][j]$: i = sensor, j = hora
- Posições percorridas: $5 \times 24 = 120$
- Relação: operações = linhas × colunas $\rightarrow O(m \times n)$

Parte 6: Análise e Conclusão

1) **Tamanho influencia operações?** Sim, diretamente.

2) **Bubble e Quick crescem igual?** Não: Bubble é quadrático, Quick é quase linear.

3) **Por que só o resultado final não basta?** Dois algoritmos podem gerar o mesmo resultado com custos muito diferentes; é preciso medir o processo (operações), não só o resultado.

Conclusão: os experimentos confirmam a teoria, algoritmos $O(n^2)$ pioram rapidamente com o aumento da entrada, enquanto $O(n \log n)$ escala de forma muito mais eficiente.